

OS concurrency Test

Identify the process concurrency
and synchronization issues in given
scenario

- **Process A** wants to use Resource X and Resource Y. It first acquires Resource X.
- **Process B** also wants to use Resource X and Resource Y. It sees that Resource X is already taken, so it acquires Resource Y instead.
- **Process A** detects that Process B has acquired Resource Y and, to avoid a deadlock, releases Resource X, thinking it will retry later.
- **Process B** detects that Process A has acquired Resource X and, to avoid a deadlock, releases Resource Y.
- **Process A** again tries to acquire Resource X, and Process B tries to acquire Resource Y. Steps 1-4 repeat.

Example Scenario: Two Processes and Two Resources

- Consider two processes, **Process A** and **Process B**, and two resources, **Resource 1** and **Resource 2**.
- **Process A** needs both Resource 1 and Resource 2 to complete its task.
- **Process B** also needs both Resource 1 and Resource 2 to complete its task.

Sequence of Events:

- **Process A** acquires **Resource 1**.
- **Process B** acquires **Resource 2**.
- **Process A** tries to acquire **Resource 2**, but it's held by **Process B**, so **Process A** is now **waiting** for Resource 2.
- **Process B** tries to acquire **Resource 1**, but it's held by **Process A**, so **Process B** is now **waiting** for Resource 1.

- **Process A** wants to write data to the buffer. It checks if the buffer is available (i.e., not being modified by Process B).
- At the same time, **Process B** wants to write data to the buffer. It also checks if the buffer is available.
- Both processes detect that the other might be trying to access the buffer, so they both attempt to back off and wait for a moment, without making any actual changes.
- After backing off, both processes check the buffer again, see that the other is still active, and back off once more.

consider two processes, **Process A** and **Process B**, trying to access and update a shared resource—a **bank account balance**—at the same time.

- **Process A** is trying to deposit \$100.
- **Process B** is trying to withdraw \$50.

The shared resource here is the **balance** variable, representing the current amount in the account.

Initial Conditions:

- The initial balance in the account is \$1000.

Scenario:

- **Process A** reads the balance (balance = \$1000).
- **Process B** reads the balance (balance = \$1000).
- **Process A** adds \$100 to the balance (new balance = \$1100), but it hasn't written this updated value to the account yet.
- **Process B**, still holding the initial balance of \$1000, subtracts \$50 (new balance = \$950), but it also hasn't written this updated value yet.
- **Process A** writes its new balance of \$1100 to the account.
- **Process B** writes its new balance of \$950 to the account.

Scenario:

- **Process A** needs to use the **printer** first, then the **scanner**.
- **Process B** needs to use the **scanner** first, then the **printer**.

Sequence of Events :

- **Process A** acquires the **printer**.
- **Process B** acquires the **scanner**.
- **Process A** tries to acquire the **scanner**, but it's held by **Process B**, so **Process A** is now waiting for the scanner.
- **Process B** tries to acquire the **printer**, but it's held by **Process A**, so **Process B** is now waiting for the printer.

Example Scenario: Shared Counter Increment

- **Process A** and **Process B** are both trying to increment a **shared counter**.
- The shared counter is initially set to **0**.

Scenario:

- **Process A** reads the counter (current value = 0).
- **Process B** reads the counter (current value = 0).
- **Process A** increments the counter (new value = 1), but it hasn't written this value back to memory yet.
- **Process B** increments the counter (new value = 1), but it also hasn't written its value back yet.
- **Process A** writes its incremented value (1) back to memory.
- **Process B** writes its incremented value (1) back to memory.

Result:

- The final counter value is **1**, which is incorrect. It should have been **2** since both processes incremented the counter.

What is the solution for this?

Scenario: Bank Account Balance Transfer

- Suppose **Process A** and **Process B**, are performing money transfers between two shared bank accounts, **Account X** and **Account Y**. Each process wants to transfer an amount from **Account X** to **Account Y**. The total balance across both accounts must remain consistent after both processes have executed.

Sequence of Events:

- **Process A** reads the balance of **Account X**.
- **Process B** reads the balance of **Account X** at the same time.
- **Process A** deducts the transfer amount from **Account X** and updates the balance.
- **Process B** also deducts the transfer amount from **Account X**, based on the initial balance it read, and updates the balance, unaware that **Process A** has already modified the balance.
- As a result, the final balance of **Account X** is incorrect, leading to data inconsistency.